

Subject

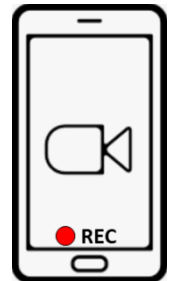
Speech and Natural Language Processing

Professor

Isaac González



Aula Virtual U/E



Participar



Session 03

Unit I

Natural Language Procesing

Topic 3

**Text preprocessing:
tokenization, stemming, lemmatization, stopwords removal**

Let's review:
S02_3_Exercises_before_solution

Please do:
S03_1_Exercises_before

Words and Tokens

Rule-based tokenization
and
Simple Unix tools

Rule-based tokenization

- Although subword tokenization is the norm
- Sometimes we need particular tokens
- Like for parsing, where the parser needs grammatical words, or social science

Issues for rule-based tokenization

- Mostly but not always remove punctuation:
 - m.p.h., Ph.D., AT&T, cap'n
 - prices (\$45.55)
 - dates (01/02/06)
 - URLs (<http://www.stanford.edu>)
 - hashtags (#nlproc)
 - email addresses (someone@cs.colorado.edu)
- Numbers are tokenized differently across languages
 - English 555,500.50 = French 555 500,50
- Multiword expressions (MWE)?
 - New York, rock 'n' roll

Penn Treebank Tokenization Standard

Input: "The San Francisco-based restaurant," they said,
"doesn't charge \$10".

Output: "_The_San_Francisco-based_restaurant_,_"_they_said_,_
"_does_n't_charge_\$_10_"_.

Tokenization in NLTK

Bird, Loper and Klein (2009), *Natural Language Processing with Python*. O'Reilly

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)          # set flag to allow verbose regexps
...     (?:[A-Z]\.)+           # abbreviations, e.g. U.S.A.
...     | \w+(?:-\w+)*         # words with optional internal hyphens
...     | \$?\d+(?:\.\d+)?%?    # currency, percentages, e.g. $12.40, 82%
...     | \.\.\.              # ellipsis
...     | [][.,;"'?:_`'-]     # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Sentence Segmentation

!, ? mostly unambiguous but **period** “.” is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.

Space-based tokenization

- A very simple way to tokenize
 - For languages that use space characters between words
 - Arabic, Cyrillic, Greek, Latin, etc., based writing systems
 - Segment off a token between instances of spaces
- Unix tools for space-based tokenization
 - The "tr" command
 - Inspired by Ken Church's UNIX for Poets
 - Given a text file, output the word tokens and their frequencies

Simple Tokenization in UNIX

- (Inspired by Ken Church's UNIX for Poets.)
- Given a text file, output the word tokens and their frequencies

Change all non-alpha to newlines

```
tr -sc 'A-Za-z'
```

Sort in alphabetical order

```
| sort
```

Merge and count each type

```
| uniq -c
```

```
1945 A
 72 AARON
 19 ABBESS      25 Aaron
  5 ABBOT       6 Abate
... ..        1 Abates
                5 Abbess
                6 Abbey
                3 Abbot
... ..
```

The first step: tokenizing

```
tr -sc 'A-Za-z' '\n' < shakes.txt | head
```

THE

SONNETS

by

William

Shakespeare

From

fairest

creatures

We

...

The second step: sorting

```
tr -sc 'A-Za-z' '\n' < shakes.txt | sort | head
```

A

A

A

A

A

A

A

A

A

...

More counting

- Merging upper and lower case

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c
```

- Sorting the counts

```
tr 'A-Z' 'a-z' < shakes.txt | tr -sc 'A-Za-z' '\n' | sort | uniq -c | sort -n -r
```

```
23243 the
22225 i
18618 and
16339 to
15687 of
12780 a
12163 you
10839 my
10005 in
8954 d
```

What happened here?

Words and Tokens

Rule-based tokenization
and
Simple Unix tools

Words
and
Tokens

Byte Pair Encoding

The NLP standard for tokenization

Instead of

- white-space / orthographic words
 - Lots of languages don't have them
 - The number of words grows without bound
- Unicode characters
 - Too small as tokens for many purposes
- morphemes
 - Very hard to define

We use the data to tell us how to tokenize.

Why tokenize?

Using a deterministic series of tokens means systems can be compared equally

- Systems agree on the length of a string

Algorithms like perplexity assume all texts have a fixed tokenization

Eliminates the problem of unknown words

If some word occurs in test set but not training set, we still know how to segment it into known tokens.

Subword tokenization

- Two most common algorithms:
 - **Byte-Pair Encoding (BPE)** (Sennrich et al., 2016)
 - **Unigram language modeling tokenization** (Kudo, 2018) (sometimes confusingly called "SentencePiece" after the library it's in)
- All have 2 parts:
 - A token **learner** that takes a raw training corpus and induces a vocabulary (a set of tokens).
 - A token **encoder/segmenter** that takes a raw test sentence and tokenizes it according to that vocabulary

Byte Pair Encoding (BPE) token learner

Iteratively merge frequent neighboring tokens to create longer tokens.

Repeat:

- Choose most frequent neighboring pair ('A', 'B')
 - Add a new merged symbol ('AB') to the vocabulary
 - Replace every 'A' 'B' in the corpus with 'AB'.
- Until k merges

Vocabulary

[A, B, C, D, E]

[A, B, C, D, E, AB]

[A, B, C, D, E, AB, CAB]

Corpus

A B D C A B E C A B

AB D C AB E C AB

AB D CAB E CAB

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

Byte Pair Encoding (BPE) Addendum

- Generally run **within** space-separated words
- Don't merge across word boundaries
 - First separate corpus by whitespace or similar, using specialized regular expressions
 - This gives a set of starting strings, with whitespace attached to start of each string
 - Counts come from the corpus, but can only merge within strings.

BPE token learner

Original (very fascinating 🤖) corpus:

set_new_new_renew_reset_renew

Put space token at start of words

corpus

2 _ n e w

2 _ r e n e w

1 s e t

1 _ r e s e t

vocabulary

_, e, n, r, s, t, w

BPE token learner

corpus

```
2  _ n e w
2  _ r e n e w
1  s e t
1  _ r e s e t
```

vocabulary

```
_, e, n, r, s, t, w
```

Merge **n e** to **ne** (count 4 = 2 **new** + 2 **renew**)

corpus

```
2  _ ne w
2  _ r e ne w
1  s e t
1  _ r e s e t
```

vocabulary

```
_, e, n, r, s, t, w, ne
```

BPE token learner

corpus

```
2  _ ne w
2  _ r e ne w
1  s e t
1  _ r e s e t
```

vocabulary

_, e, n, r, s, t, w, ne

Merge **ne w** to **new** (count 4)

corpus

```
2  _ new
2  _ r e new
1  s e t
1  _ r e s e t
```

vocabulary

_, e, n, r, s, t, w, ne, new

BPE token learner

corpus

2 _ new

2 _ r e new

1 s e t

1 _ r e s e t

vocabulary

_, e, n, r, s, t, w, ne, new

Merge _ r to _r (count 3) and _r e to _re (count 3)

corpus

2 _ new

2 _re new

1 s e t

1 _re s e t

vocabulary

_, e, n, r, s, t, w, ne, new, _r, _re

•System has learned prefix re- !

BPE

The next merges are:

merge	current vocabulary
(-, new)	-, e, n, r, s, t, w, ne, new, -r, -re, -new
(-re, new)	-, e, n, r, s, t, w, ne, new, -r, -re, -new, -renew
(s, e)	-, e, n, r, s, t, w, ne, new, -r, -re, -new, -renew, se
(se, t)	-, e, n, r, s, t, w, ne, new, -r, -re, -new, -renew, se, set

BPE encoder algorithm

Tokenize a test sentence: run each merge learned from the training data:

- Greedily, in the order we learned them
- (test frequencies don't play a role)

First: segment each test word into characters

Then run rules: (1) merge every **n e** to **ne**, (2) merge **ne w** to **new**, (3) **_r**, (4) **_re** etc.

•Result:

- Recreates training set words
- But also learns subwords like **_re** that might appear in new words like **rearrange**

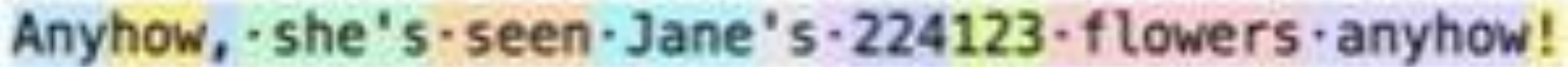
BPE and Unicode

We run BPE on large Unicode corpora, with vocabulary sizes of 50,000 to 200,000

- On individual bytes of UTF-8-encoded text
 - Not on Unicode characters
 - BPE rediscovers 2-byte and common 3-byte UTF-8 sequences
 - Only 256 possible values of a byte, so no unknown tokens
 - (BPE might learn a few illegal UTF-8 sequences across character boundaries, but these can be filtered)

Visualizing GPT4o tokens

Tat Dat Duong's [Tiktokenizer](#) visualizer



Anyhow, -she's-seen-Jane's-224123-flowers-anyhow!

- Tokens: 11865, 8923, 11, 31211, 6177, 23919, 885, 220, 19427, 7633, 18887, 147065, 0
- Most words are tokens, w/initial space
- Clitics like 's
 - Are segmented off Jane
 - But part of frequent words like she's
- Numbers segmented into chunks of 3 digits
- Anyhow and -anyhow are segmented differently
- Some of this is from preprocessing
 - regular expressions for chunking digits, stripping clitics

Tokenizing across languages

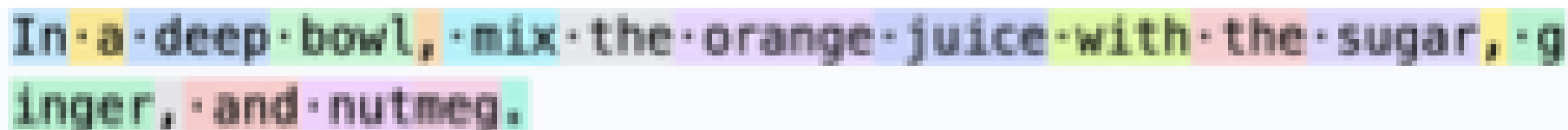
- Even though BPE tokenizers are multilingual
- LLM training data is still vastly dominated by English
 - Most BPE tokens used for English, leaving less for other languages
 - Words in other languages are often split up

Tokenization is better in English

Tat Dat Duong's [Tiktokenizer](#) visualizer on GPT4o

- A recipe sentence in two languages

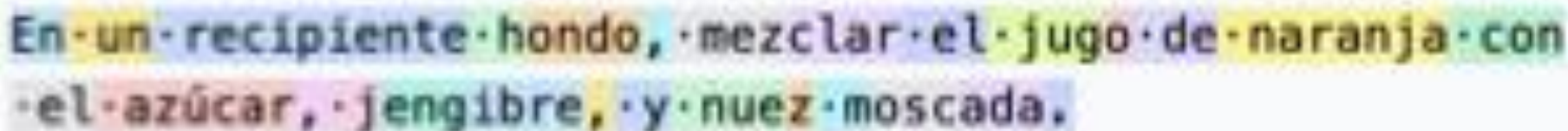
English: 18 tokens; no words are split into multiple tokens):



In·a·deep·bowl,·mix·the·orange·juice·with·the·sugar,·ginger,·and·nutmeg.

This visualization shows the English sentence "In a deep bowl, mix the orange juice with the sugar, ginger, and nutmeg." broken down into 18 individual tokens. Each token is represented by a colored rectangular block, and the tokens are separated by a dot (·). The tokens are: In, a, deep, bowl, , mix, the, orange, juice, with, the, sugar, , ginger, , and, nutmeg, .

Spanish: 33 tokens; 6/16 words are split



En·un·recipiente·hondo,·mezclar·el·jugo·de·naranja·con·el·azúcar,·jengibre,·y·nuez·moscada.

This visualization shows the Spanish sentence "En un recipiente hondo, mezclar el jugo de naranja con el azúcar, jengibre, y nuez moscada." broken down into 33 individual tokens. Each token is represented by a colored rectangular block, and the tokens are separated by a dot (·). The tokens are: En, un, recipiente, hondo, , mezclar, el, jugo, de, naranja, con, el, azúcar, , jengibre, , y, nuez, moscada, .

Words
and
Tokens

Byte Pair Encoding

Words
and
Tokens

Pretokenization for BPE

Pretokenization for BPE

- The GPE-2 rules for pretokenization:
 - Count clitics like 't and 'm as tokens
 - Pull off punctuation and numbers
- We do this with one regex:

```
r"'s|'t|'re|'ve|'m|'ll|'d| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!
```

The Python regex (as opposed to re) library

- Special `\p` and `\P` operators:
- `\p{L}` matches any Unicode letter,
- `\P{L}` matches any non-letter,
- `\p{N}` matches any number,
- `\P{N}` matches any non-number.

```
>>> import regex as re
>>> pat = re.compile(
...     # Contractions: 't and 'm are tokens
...     r"'s|'t|'re|'ve|'m|'ll|'d|"
...     # Words:  sequence of Unicode letters (after optional space)
...     r" ?\p{L}+|"
...     #Number: sequence of digits (after optional space)
...     r" ?\p{N}+|"
...     # Punctuation: sequence of non-alphanumeric/non-space
...                       #(after optional space)
...     r" ?[^\s\p{L}\p{N}]+|"
...     # whitespace
...     r"\s+(?!\\S)|\s+"
... )
>>> text = "We're 350 dogs! Um, lunch?"
>>> print(pat.findall(text))
['We', "'re", ' 350', ' dogs', '!', ' Um', ',,', ' lunch', '?']
>>>
```

Pretokenization could also allow multiple words!

SUPERBPE runs a second stage of BPE allowing merges across spaces and punctuation.

BPE: By the way, I am a fan of the Milky Way.

SuperBPE: By the way, I am a fan of the Milky Way.

Words
and
Tokens

Pretokenization for BPE

Basic Text Processing

Word Normalization and
other issues

Word Normalization

- Putting words/tokens in a standard format
 - U.S.A. or USA
 - uhhuh or uh-huh
 - Fed or fed
 - am, is, be, are

Case folding

- Applications like IR: reduce all letters to lower case
 - Since users tend to use lower case
 - Possible exception: upper case in mid-sentence?
 - e.g., ***General Motors***
 - ***Fed*** vs. ***fed***
 - ***SAIL*** vs. ***sail***
- For sentiment analysis, MT, Information extraction
 - Case is helpful (***US*** versus ***us*** is important)

Lemmatization

Represent all words as their lemma, their shared root
= dictionary headword form:

- *am, are, is* → *be*
- *car, cars, car's, cars'* → *car*
- Spanish **quiero** ('I want'), **quieres** ('you want')
→ **querer** 'want'
- *He is reading detective stories*
→ *He be read detective story*

Lemmatization is done by Morphological Parsing

- Morphemes:

- The small meaningful units that make up words
- **Stems**: The core meaning-bearing units
- **Affixes**: Parts that adhere to stems, often with grammatical functions

- Morphological Parsers:

- Parse *cats* into two morphemes *cat* and *s*
- Parse Spanish *amaren* ('if in the future they would love') into morpheme *amar* 'to love', and the morphological features *3PL* and *future subjunctive*.

Stemming

- Reduce terms to stems, chopping off affixes crudely

This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.



Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note .

Porter Stemmer

- Based on a series of rewrite rules run in series
 - A cascade, in which output of each pass fed to next pass
- Some sample rules:

ATIONAL → ATE (e.g., relational → relate)

ING → ε if stem contains vowel (e.g., motoring → motor)

SSES → SS (e.g., grasses → grass)

Dealing with complex morphology is necessary for many languages

- e.g., the Turkish word:
- **Uygarlastiramadiklarimizdanmissinizcasina**
- ‘(behaving) as if you are among those whom we could not civilize’
- **Uygar** ‘civilized’ + **las** ‘become’
 - + **tir** ‘cause’ + **ama** ‘not able’
 - + **dik** ‘past’ + **lar** ‘plural’
 - + **imiz** ‘p1pl’ + **dan** ‘abl’
 - + **mis** ‘past’ + **siniz** ‘2pl’ + **casina** ‘as if’

Sentence Segmentation

!, ? mostly unambiguous but **period** “.” is very ambiguous

- Sentence boundary
- Abbreviations like Inc. or Dr.
- Numbers like .02% or 4.3

Common algorithm: Tokenize first: use rules or ML to classify a period as either (a) part of the word or (b) a sentence-boundary.

- An abbreviation dictionary can help

Sentence segmentation can then often be done by rules based on this tokenization.

Basic Text Processing

Word Normalization and
other issues

Please redo with unix tools, NLTK, and Spacy.
Use several methods if aplicable.
S03_1_Exercises_before

Please do:
S03_3_Exercises